JARVIS

Just A Rather Very Intelligent System

Personal AI Voice Assistant — Project Report

Submitted by: Hussnain
Institution: UET Techslab — Cybersecurity Department
Date: June 2026

1. Introduction

JARVIS (Just A Rather Very Intelligent System) is a Python-based voice-controlled personal assistant developed as a practical software project. Inspired by the fictional AI assistant in the Iron Man universe, this project aims to bring similar functionality to a real desktop environment using modern open-source tools.

The assistant runs entirely on the user's local machine, combining real-time speech recognition, text-to-speech output, system automation, and an on-device large language model (LLM) to deliver an intelligent, privacy-respecting experience without any cloud subscription or external AI API dependency.

2. Project Objectives

The primary objectives of this project are:

- Develop a fully functional voice-controlled assistant that responds to natural language commands.
- Integrate a locally hosted AI model (Llama 3.1 via Ollama) for open-ended question answering.
- Enable hands-free control of PC operations including app launching, volume, screenshots, and power actions.
- Provide real-time information such as weather, time, date, and web search results.
- Build a modular, extensible codebase that can easily be expanded with new features.

3. System Overview

JARVIS follows a simple listen → process → respond pipeline. The system continuously listens for voice input, interprets the command through keyword matching or AI inference, executes the corresponding action, and speaks the result back to the user.

3.1 Architecture

Component	Technology	Purpose
Speech Input	SpeechRecognition + Google API	Converts microphone audio to text
Text Output	pyttsx3	Converts text responses to speech
AI Brain	Ollama + Llama 3.1 (8B)	Answers open-ended questions locally

Component	Technology	Purpose
Web Data	requests + wttr.in	Fetches weather and news
PC Control	os, subprocess, pyautogui	Controls system and applications
Web Actions	webbrowser module	Opens URLs and performs searches

4. Features & Capabilities

4.1 Voice Recognition

JARVIS uses the SpeechRecognition library with Google's Speech-to-Text engine for accurate voice input. The recognizer is tuned with an energy threshold of 3000 and dynamic energy threshold adjustment to handle varying ambient noise conditions. A 6-second timeout and 10-second phrase limit ensure responsive listening.

4.2 Application Launcher

With a single voice command, JARVIS can launch a wide range of applications including Chrome, Notepad, VS Code, Calculator, Paint, Command Prompt, VMware Workstation, Windows Camera, Task Manager, and File Explorer.

4.3 Web Navigation & Search

JARVIS supports hands-free web interaction including Google search, YouTube search, Wikipedia lookups, and direct navigation to platforms such as Instagram, GitHub, Gmail, Facebook, and Twitter.

4.4 Real-Time Information

Live weather data is fetched for any specified city using the wttr.in API. The assistant also reports the current time and date in a natural, spoken format.

4.5 PC Control

JARVIS can control system-level functions entirely by voice, including volume adjustment (up, down, mute), screenshot capture with automatic timestamped filenames, screen lock, sleep, system shutdown, and restart.

4.6 AI-Powered Q&A

For any unrecognized command or knowledge question, JARVIS forwards the query to a locally running Llama 3.1 (8B) model via the Ollama API. Responses are trimmed to 2-3 sentences for concise, spoken delivery. This ensures JARVIS never leaves a question unanswered.

5. Implementation Details

5.1 Development Environment

Item	Details
Language	Python 3.x
IDE	Visual Studio Code
Hardware	HP EliteBook x360 — 32GB RAM, 512GB SSD
OS	Windows 11
AI Runtime	Ollama (local LLM server)
AI Model	Meta Llama 3.1 — 8 Billion Parameters

5.2 Key Libraries

Library	Version	Usage
SpeechRecognition	3.x	Microphone audio to text conversion
pyttsx3	2.x	Offline text-to-speech engine
requests	2.x	HTTP calls to Ollama API and wttr.in
pyautogui	0.9.x	Volume keys and screenshot capture
webbrowser	Built-in	Opening URLs in the default browser
os / subprocess	Built-in	Launching executables and system commands

5.3 Command Processing Logic

All commands are handled inside the run_command() function using a chain of elif statements with keyword matching. This approach makes the logic transparent and easy to extend. Commands that do not match any keyword are automatically routed to the Ollama AI fallback, ensuring comprehensive coverage of user intents.

6. Sample Voice Commands

Voice Command	Action Performed
"Open Chrome"	Launches Google Chrome browser
"Weather in Karachi"	Fetches and speaks live weather for Karachi

Voice Command	Action Performed
"What time is it"	Announces the current system time
"Search machine learning"	Opens Google search results
"YouTube search lofi music"	Searches YouTube for lofi music
"Take a screenshot"	Saves a timestamped PNG screenshot
"Volume up"	Increases system volume by 5 steps
"Lock my PC"	Locks the Windows workstation
"What is quantum computing"	Queries Llama 3.1 and speaks the answer
"Shutdown"	Initiates system shutdown after 5 seconds
"Tell me a joke"	Speaks a random programming joke
"Goodbye JARVIS"	Exits the assistant gracefully

7. Challenges & Solutions

Challenge	Solution Applied
Voice recognition accuracy in noisy environments	Enabled dynamic energy threshold and ambient noise adjustment
Ollama response latency on large queries	Limited AI responses to 2-3 sentences via prompt engineering
TTS voice quality	Configured pyttsx3 rate and voice properties for natural delivery
Handling unrecognized commands gracefully	Routed all fallback inputs to the local Llama model
PC automation compatibility	Used os.system, subprocess.Popen, and pyautogui for broad Windows support

8. Future Improvements

- Wake word detection (e.g. "Hey JARVIS") to avoid continuous listening.
- Faster, more accurate offline speech recognition using Whisper by OpenAI.
- Improved TTS voice quality using a neural TTS engine such as Coqui TTS.
- GUI dashboard to display assistant status, command history, and settings.
- Multi-language support including Urdu for broader accessibility.
- Smart home device integration via IoT APIs.
- Memory system to remember user preferences across sessions.

9. Conclusion

JARVIS demonstrates the practical integration of speech recognition, system automation, REST APIs, and on-device AI into a cohesive, always-available desktop assistant. Built entirely with open-source Python tools, it achieves a balance between functionality and privacy by keeping all AI inference local.

The project successfully meets its core objectives and provides a strong foundation for future development. It showcases skills in Python programming, API integration, system-level scripting, and AI model deployment — all highly relevant to a cybersecurity and software engineering background.

References

1. Python SpeechRecognition Library — https://pypi.org/project/SpeechRecognition/
2. pyttsx3 Text-to-Speech — https://pypi.org/project/pyttsx3/
3. Ollama Local LLM Runtime — https://ollama.ai
4. Meta Llama 3.1 Model — https://ai.meta.com/llama/
5. wttr.in Weather API — https://wttr.in
6. PyAutoGUI Documentation — https://pyautogui.readthedocs.io